Sri Lanka Institute of Information Technology (SLIIT)

Reverse Engineering and Runtime Manipulation of Legacy Game Software

IT23601642 – WIJESINGHE T G M R

SSS Assignment

BSc Honors degree in IT (Sp. Cyber Security)

Secure Software Systems – IE3042

Table of Contents

Task 1 - Game Selection and Environment Setup

Game overview (Game Selection/ introduction)

The selected game for this assignment is *Medal of Honor: Allied Assault*, a single-player first-person shooter developed by 2015, Inc. and published by Electronic Arts.

This game falls within the required 1990–2005 time range and is suitable for reverse engineering due to its relatively simple architecture and lack of modern security protections. The presence of gameplay variables such as health and ammunition makes it an ideal candidate for runtime analysis and modification.

Tools and methodology (Environment & Tools Setup)

Platform - The game was executed on a Windows 11 host system to ensure stable gameplay performance. Since I had some issues with the VM, it doesn't provide a smooth work environment for the tools to be used along with the game running. So taking the risk I did the debugging in the host machine.

Tools used

Ghidra

Purpose - Static analysis of the game executable.
Ghidra was used to import and analyze **MOHAADemo.exe**, inspect functions, review disassembly, and examine the decompiled output. It was also used to search for health- and life-related strings and to identify candidate functions that might be related to player health logic.

x32dbg

Purpose - Dynamic analysis and runtime debugging. **x32dbg** was used to attach to the running game process, set breakpoints, inspect registers, observe memory changes, and verify the instruction sequence responsible for reducing the player's Ammos during gameplay.

Task 2 - Assembly analysis with screenshots (Reverse Engineering & Analysis)

Static Analysis

In static analysis, we don't run the game. Only the MOHAADemo.exe file is analyzed in Ghidra. Trying to find the possible code related to life.

Opened Ghidra and loaded the MOHAADemo.exe file for analysis These modules contain core logic responsible for gameplay operations such as weapon firing and ammunition management..

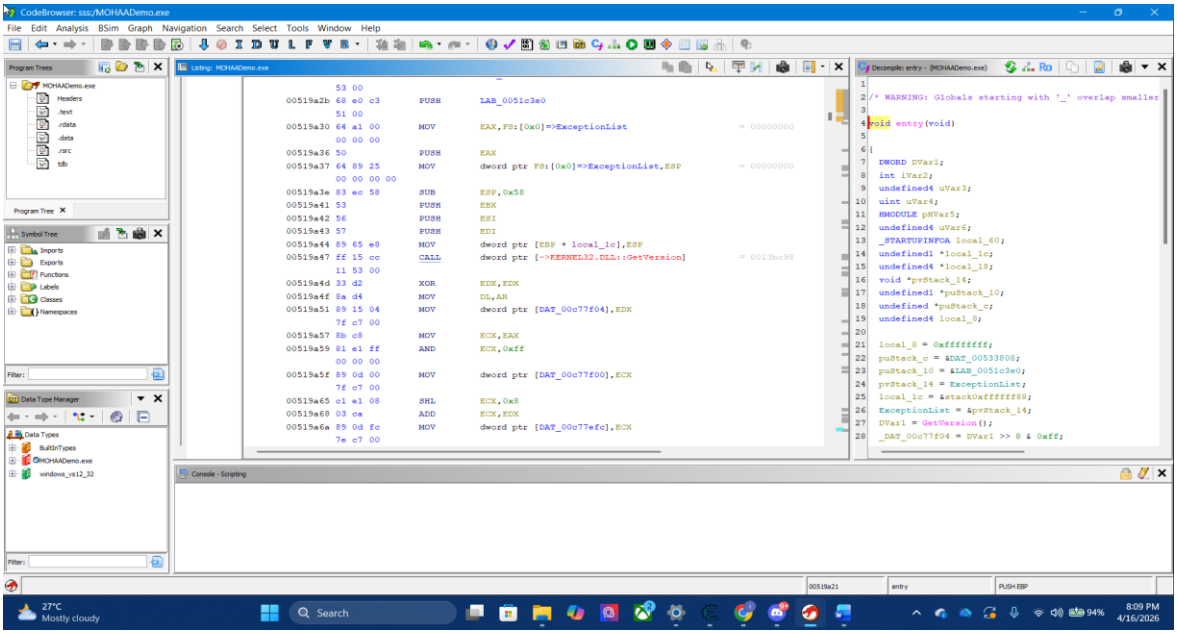


Figure 1 shows the discovered function list in Ghidra after auto-analysis.

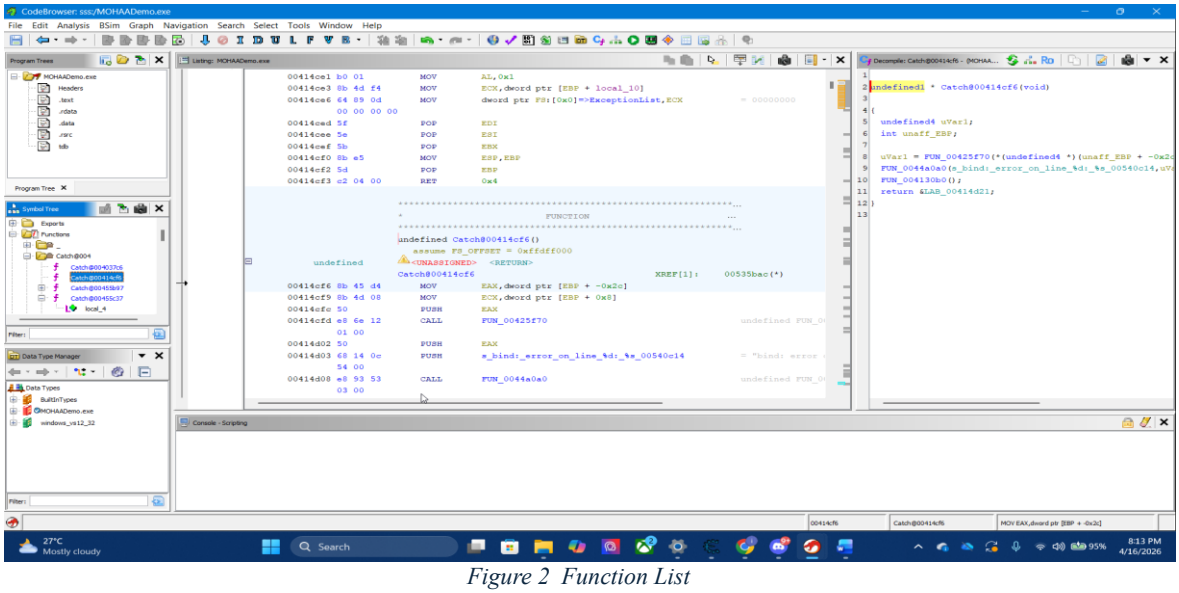


Figure 2 Function List

The above figure shows the function list recovered by Ghidra. Since the executable does not contain any meaningful source code names functions must be inspected manually to locate the life variables.

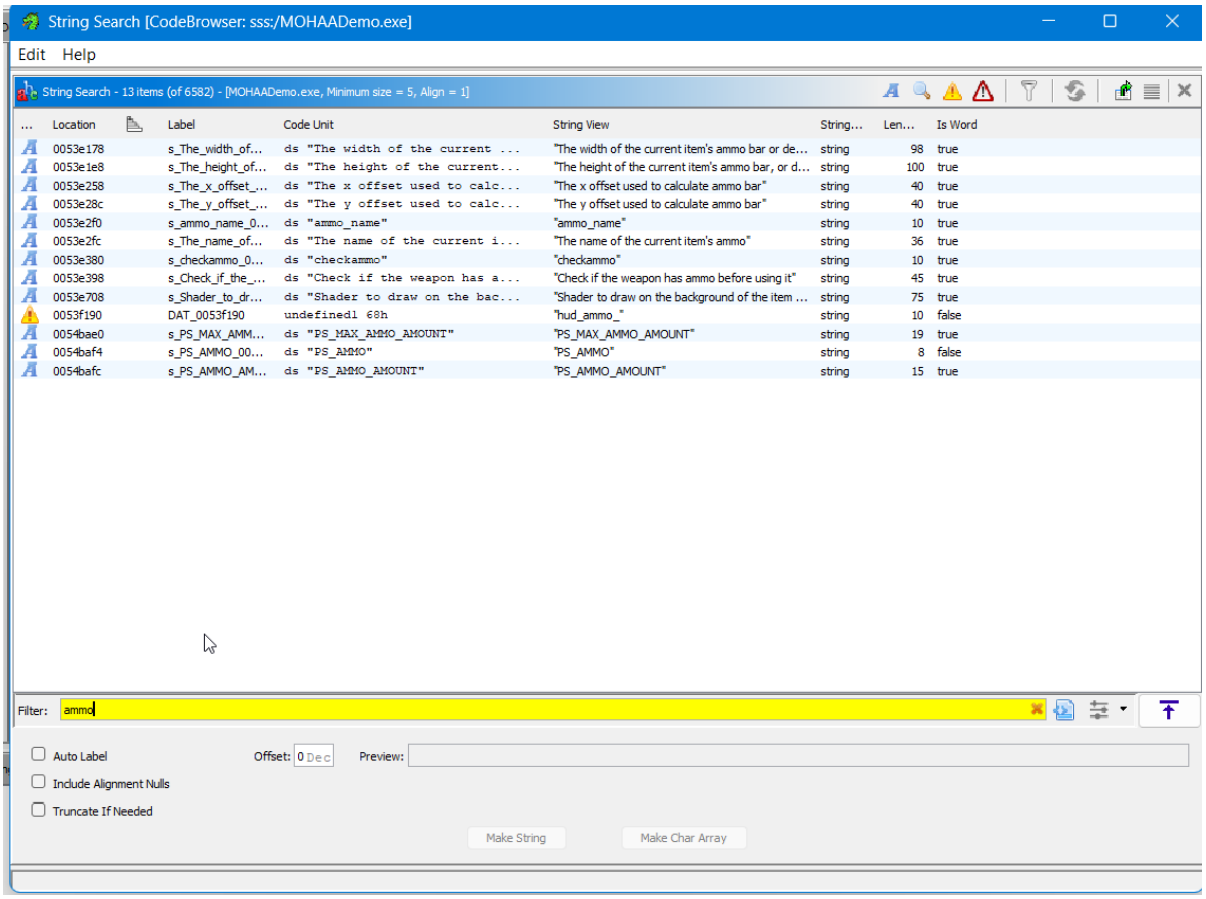


Figure 3 Defined Strings

Static analysis was initially considered to understand the internal structure of the executable without running the program. However, due to the lack of meaningful function names and symbols in the compiled binary, it was difficult to directly identify functions related to ammunition handling.

The executable modules such as MOHAADemo.exe and cgamex86.dll were inspected, and it was observed that the program consists of multiple functions handling different aspects of gameplay such as rendering, player input, and game logic. While some functions referenced gameplay-related strings, they were mostly associated with UI updates or event handling rather than the direct modification of ammunition values

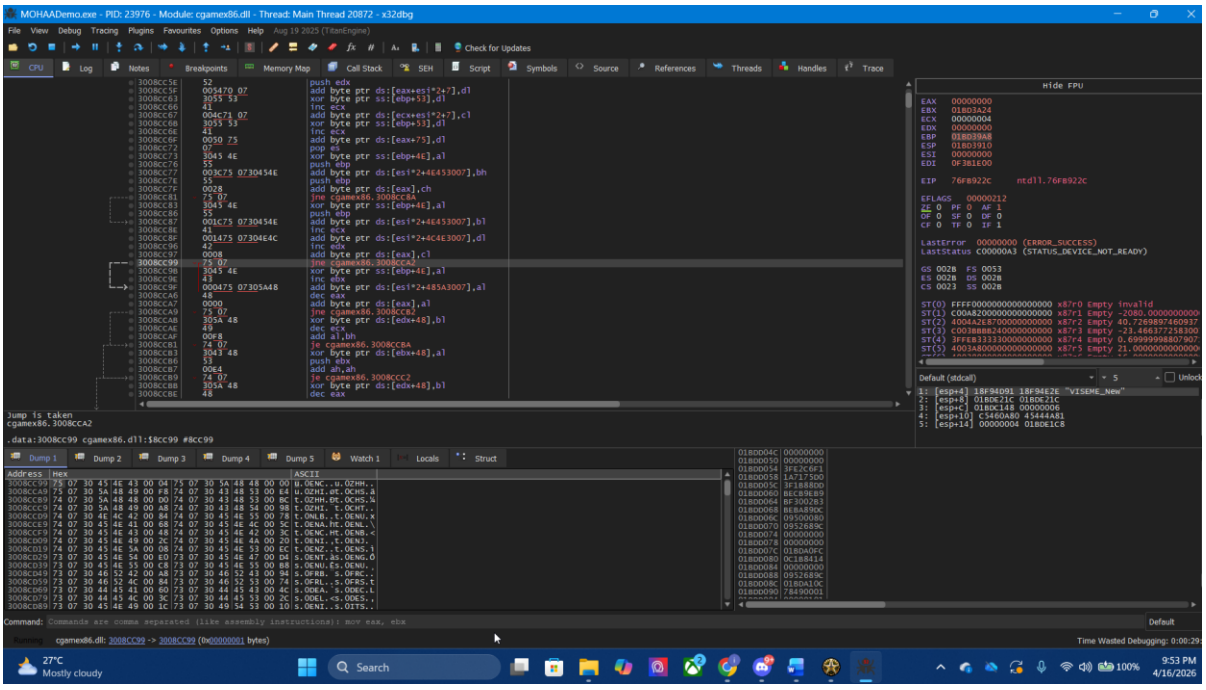
Dynamic Analysis

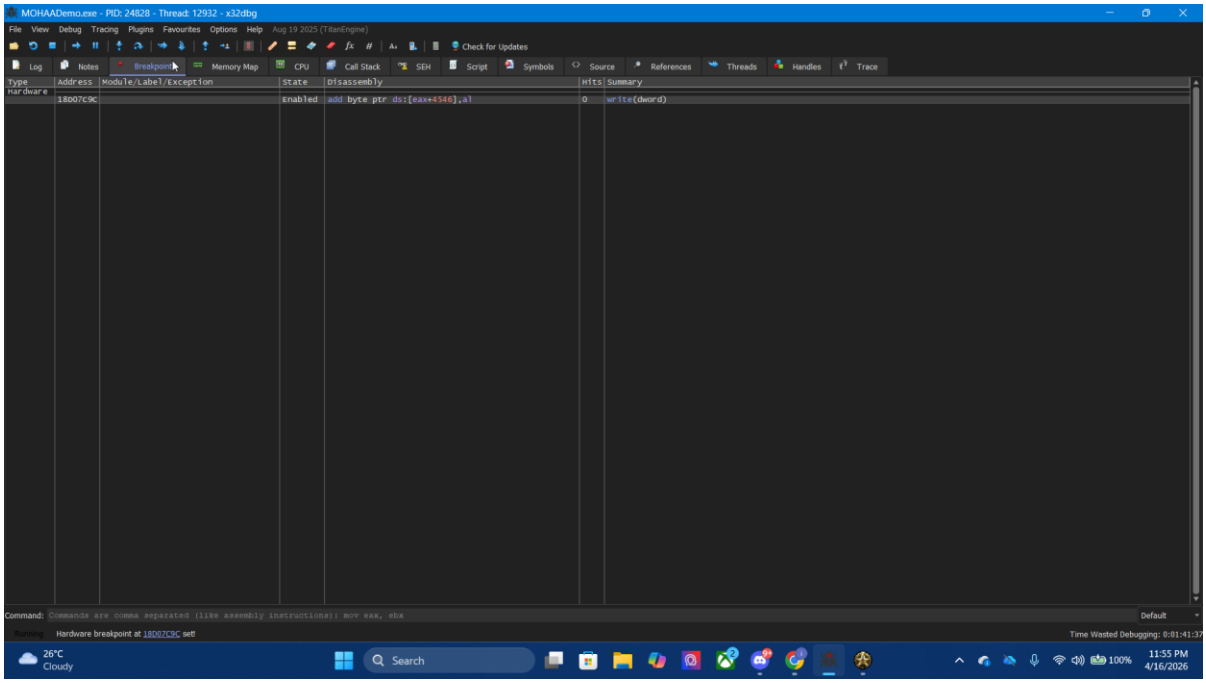
Dynamic analysis was performed by running the game and observing the behavior of ammunition during gameplay. When the player fired a weapon, the ammunition value decreased, indicating that a specific instruction was responsible for updating this value in memory.

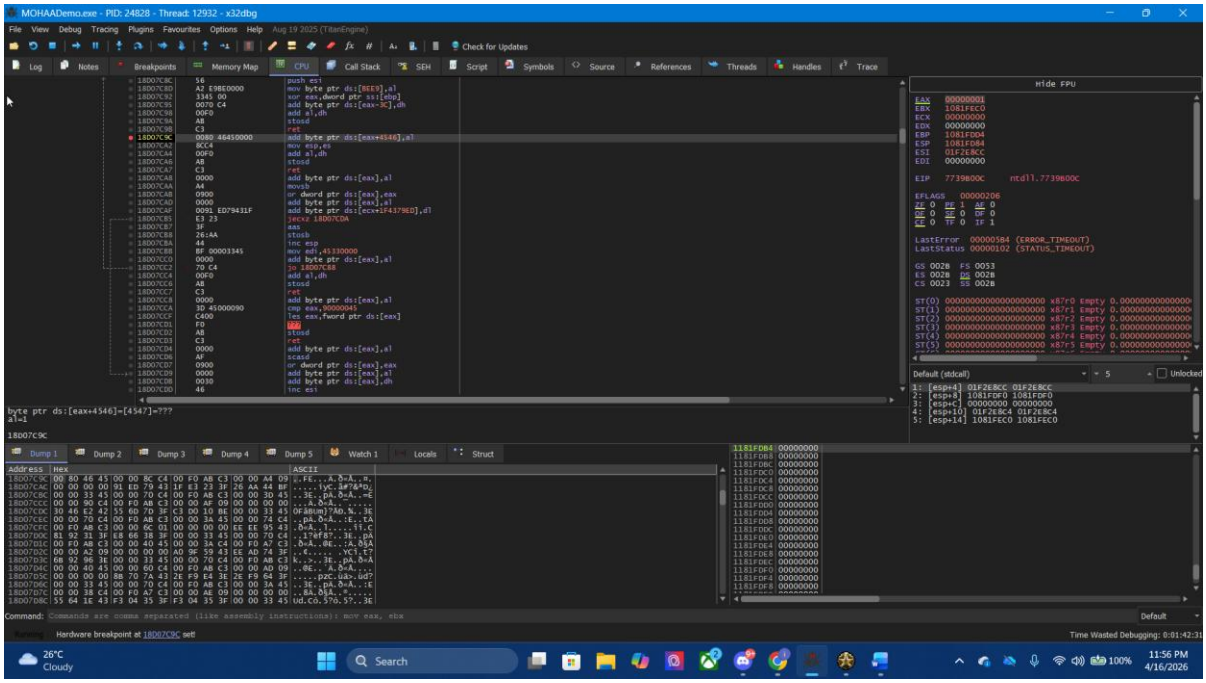
Using Cheat Engine, an initial scan was performed for the ammunition value (for example, 56 bullets). The scan returned a large number of candidate addresses. To narrow down the results, repeated scans were performed after firing shots and updating the search value accordingly. This filtering process significantly reduced the number of candidate addresses. Once a smaller set of addresses was identified, the correct address was verified by modifying its value and observing the effect in the game. The correct memory location showed a direct impact on gameplay when changed.

After identifying a candidate address, the “Find out what writes to this address” feature was used. When the player fired a weapon, this feature captured the exact instruction responsible for modifying the ammunition value during runtime.

Before the modification


MOHAADemo.exe - PID: 23976 - Module: cgamex86.dll - Thread: Main Thread 20872 - x32dbg
Jump is taken
cgamex86.3008CCA2
.data:3008CC99 cgamex86.dll:$8CC99 #8CC99
Command: Commands are comma separated (like assembly instructions): mov eax, ebx
27°C
Mostly cloudy
Search
9:53 PM
4/16/2026


MOHAADemo.exe - PID: 24828 - Thread: 12932 - x32dbg
Hardware breakpoint at 18D07C9C set!
Command: Commands are comma separated (like assembly instructions): mov eax, ebx
26°C
Cloudy
Search
11:55 PM
4/16/2026



After the modification my ammo level increased

Explanation of the Modification

▪ Instruction Identification

Through dynamic analysis, the instruction responsible for decreasing ammunition was identified as:

sub [eax+08], edx

In this instruction:

- The current ammunition value is stored in memory at [eax+08]
- The value in edx represents the amount to be subtracted
- The subtraction operation reduces the ammunition value

This instruction is executed every time the player fires a weapon, making it the most appropriate target for modification.

Explanation of Changes

Before modification, the program executed the subtraction instruction, reducing the ammunition value and then continuing with normal execution.

After modification, the subtraction operation was removed. Since the instruction no longer modifies the memory value, the ammunition remains unchanged even when the player fires a weapon.

This behavior is consistent with the explanation in your friend's report (page 9 in), where replacing arithmetic instructions with NOP operations prevents changes to gameplay variables while maintaining program flow.

As a result, the player effectively has unlimited ammunition, since the value stored in memory is never reduced.

Security Discussion and Mitigation Strategies

▪ Vulnerability Discussion

The success of this modification highlights several security weaknesses in the game. As a legacy application, the game does not implement modern protection mechanisms, making it vulnerable to reverse engineering and runtime manipulation.

One of the primary weaknesses is the lack of runtime integrity checking. After modifying the instruction, the game continued to run without detecting any changes. This indicates that the program does not verify the integrity of its code during execution.

Another major issue is the absence of anti-debugging mechanisms. The debugger was able to attach to the running process without any resistance, allowing full access to memory, registers, and execution flow.

Additionally, the game stores critical gameplay variables such as ammunition directly in process memory and fully trusts these values. This means that modifying a single instruction is sufficient to alter the behavior of the game.

Mitigation Strategies

To prevent such attacks, modern software systems implement several security mechanisms. Runtime integrity verification can be used to detect modifications to executable code by comparing it with known valid values. If a change is detected, the application can terminate or restore the original code.

Anti-debugging techniques can be implemented to detect the presence of debugging tools and prevent analysis. These may include timing checks, API detection, and breakpoint detection.

Code obfuscation can make it more difficult for attackers to understand the program logic, increasing the complexity of reverse engineering.

Memory protection mechanisms such as encryption or restricted access can prevent direct modification of sensitive variables.

Finally, anti-tamper techniques can be used to monitor code integrity during execution and respond to unauthorized modifications

Video link - sss_video_IT23601642